The `select` method lets you override the SQL `SELECT` clause that is used to retrieve data about the associated object. By default, Rails retrieves all columns.
If you use the `select` method on a `belongs_to` association, you should also set the `:foreign_key` option to guarantee the correct results.

4.1.4 Do Any Associated Objects Exist?

You can see if any associated objects exist by using the `association.nil?` method:

```
if @order.customer.nil?
  @msg = "No customer found for this order"
end
```

4.1.5 When are Objects Saved?

Assigning an object to a `belongs_to` association does *not* automatically save the object. It does not save the associated object either.

4.2 has_one Association Reference

The `has_one` association creates a one-to-one match with another model. In database terms, this association says that the other class contains the foreign key. If this class contains the foreign key, then you should use `belongs_to` instead.

4.2.1 Methods Added by has_one

When you declare a `has_one` association, the declaring class automatically gains five methods related to the association:

- `association(force_reload = false)`
- `association=(associate)`
- `build_association(attributes = {})`
- `create_association(attributes = {})`
- `create_association!(attributes = {})`

In all of these methods, `association` is replaced with the symbol passed as the first argument to `has_one`. For example, given the declaration:

```
class Supplier < ActiveRecord::Base
  has_one :account
end
```

Each instance of the `Supplier` model will have these methods:

```
account
account=
build_account
create_account
create_account!
```

When initializing a new `has_one` or `belongs_to` association you must use the `build_`prefix to build the association, rather than the `association.build` method that would be used for `has_many` or `has_and_belongs_to_many` associations. To create one, use the `create_` prefix.

4.2.1.1 association(force_reload = false)

The `association` method returns the associated object, if any. If no associated object is found, it returns `nil`.

```
@account = @supplier.account
```

If the associated object has already been retrieved from the database for this object, the cached version will be returned. To override this behavior (and force a database read), pass `true` as the `force_reload` argument.

4.2.1.2 association=(associate)